Python Engineering - Reference Guide

Class 06 - Dictionaries & Sets: simple reference guide

Expanded reference for **week_02_data_structures -> class_06_dictionaries_sets**. The goal is to come back months later and quickly remember what dictionaries and sets are, how they differ, how data is accessed, and when each structure is useful.

1. Dictionary: intuitive definition

A **dictionary** stores **key: value** pairs. Instead of locating data by a numeric position, as with a list, you locate it through a descriptive key.

```
config = {
    'h0': 0.0,
    'v0': 50.0,
    'g': 9.81,
    'dt': 0.1
}

config['v0']   # 50.0
config['dt']   # 0.1
```

Mental rule: list = 'give me the item at position 2'. Dictionary = 'give me the value associated with key dt'.

2. Main dictionary characteristics

Characteristic	Meaning
Key:value pairs	each piece of data is identified by a key
Unique keys	the same key cannot identify two separate entries
Mutable	entries can be added, updated, or removed
Access by key	no need to remember numeric indices
Mixed value types	numbers, strings, lists, dicts, etc.
Insertion order	modern Python preserves key insertion order

3. Create, read, and modify a dictionary

```
config = {'h0': 0.0, 'v0': 50.0}

# Read
v0 = config['v0']

# Update
config['v0'] = 60.0

# Add
config['dt'] = 0.1

# Delete
del config['h0']
```

4. Safe access: [] versus .get()

```
config['dt']
# KeyError if missing

config.get('dt')
# None if missing

config.get('dt', 0.01)
# returns 0.01 if missing
```

`d[key]` is ideal when the key must exist. `d.get(key)` is useful when the key may be absent.

5. Important dict methods and functions

Method / function	What it does	Example
d.keys()	view of keys	config.keys()
d.values()	view of values	config.values()
d.items()	(key, value) pairs	config.items()
d.get(k)	safe key access	config.get('dt')
d.update({...})	update several entries	config.update({'dt':0.01})
d.pop(k)	remove and return entry	config.pop('dt')
d.clear()	empty the dictionary	config.clear()
len(d)	number of entries	len(config)
k in d	check whether a key exists	'dt' in config

6. Iterating over a dictionary

```
for key in config:
    print(key)

for value in config.values():
    print(value)

for key, value in config.items():
    print(key, value)
```

`for key, value in d.items()` is one of the most common dictionary patterns because it exposes the data name and the data value together.

7. Nested dictionaries

```
simulation = {
    'initial_conditions': {
        'h0': 0.0,
        'v0': 50.0
    },
    'solver': {
        'method': 'Euler',
        'dt': 0.1
    }
}

simulation['solver']['dt']   # 0.1
```

8. Common dictionary use cases

Situation	Why dict is useful
Configurations	each parameter has a descriptive name
Results	each quantity has a descriptive key
Sensor data	sensor_id -> measurement
Metadata	name, units, timestamp, source...
JSON / APIs	JSON closely resembles nested dictionaries
Complex function parameters	avoids long positional argument lists

9. Set: intuitive definition

A **set** is a collection of **unique elements**. It is not designed to map names to values; it is designed for questions such as 'what do I have?', 'what is missing?', and 'what do these groups have in common?'.

```
required = {'h0', 'v0', 't0', 'g', 'dt'}
provided = {'h0', 'v0', 'g', 'dt'}

missing = required - provided
# {'t0'}
```

10. Main set characteristics

Characteristic	Meaning
Unique elements	duplicates disappear
Not indexed	set[0] does not exist
Mutable	elements can be added or removed
Fast membership checks	excellent for x in my_set
Set algebra	union, intersection, difference, etc.
No key:value pairs	a set stores elements, not mappings

11. Creating sets and removing duplicates

```
values = [1, 1, 2, 2, 3, 3]
unique_values = set(values)

print(unique_values)
# {1, 2, 3}
```

12. Main set methods

Method	What it does	Example
.add(x)	add one element	A.add('dt')
.remove(x)	remove; error if missing	A.remove('dt')
.discard(x)	remove without error if missing	A.discard('dt')
.pop()	remove an arbitrary element	A.pop()
.clear()	empty the set	A.clear()
.copy()	shallow copy	B = A.copy()
len(A)	number of elements	len(A)
x in A	membership	'dt' in A

13. Set operations

Operation	Syntax	Meaning
Union	A | B	all elements appearing in A or B
Intersection	A & B	elements common to both
Difference	A - B	elements in A but not B
Symmetric difference	A ^ B	elements in one set but not both
Subset	A <= B	all A elements are inside B

Operation	Syntax	Meaning
Superset	A >= B	A contains all B elements

14. Practical example: validating configuration

```
required = {'h0', 'v0', 't0', 'g', 'dt'}
provided = set(config.keys())

missing = required - provided

if missing:
    print('Missing keys:', missing)
else:
    print('Configuration complete')
```

This is a natural use for sets: order does not matter, only whether every required element is present.

15. Dictionary vs set: fundamental difference

Question	Dictionary	Set
What does it store?	key:value pairs	unique elements
How do I access?	by key	membership, not index
Associated values?	yes	no
Duplicates?	keys no; values yes	no
Typical use	named data	membership / comparing groups
Example	{'g':9.81}	{'h0','v0','g'}

16. Dict, set, list, or tuple?

Need	Recommended structure
Ordered sequence that changes	list
Fixed ordered state	tuple
Named values / keys	dict
Unique elements / membership	set

17. Common mistakes

```
# 1. Missing dictionary key
config['unknown']      # KeyError

# 2. Trying to index a set
A[0]                   # TypeError

# 3. Empty set
{}                     # EMPTY DICT
set()                  # empty set

# 4. Duplicate dict key
{'g': 9.81, 'g': 1.62}
# final value for 'g' is 1.62
```

Important: `{}` creates an empty dictionary, not an empty set. Use `set()` for an empty set.

18. Mini cheat sheet

```
DICT
{'key': value}          create
d['key']                read
```

```
d.get('key')            safe read
d['key'] = value        add/update
d.keys()                keys
d.values()              values
d.items()               key/value pairs
'key' in d              membership

SET
{1, 2, 3}               create
set()                   empty set
A.add(x)                add
A.remove(x)             remove
A | B                   union
A & B                   intersection
A - B                   difference
x in A                  membership
```